BABU BANARASI DAS UNIVERSITY

2026-2027

SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

Student Management System Using MongoDB

Submitted to: Mr. Harendra Singh **Submitted by:** Akshat Soni

Department: BCA (DS & AI) **Roll No:** 1250258060

Date: 25th September 2026 **University Roll No:** 1250258060

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Student Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including their roll number, name, age, marks, and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting, and searching records. Comparison operators such as $eq, $gt, $lt, $gte, and $lte have also been demonstrated.

OBJECTIVES

• To understand the concept of NoSQL databases.
• To learn the basic features of MongoDB.
• To create a database and collection.
• To insert 50 student records.
• To perform CRUD operations.
• To use comparison operators.
• To perform sorting and filtering.
• To understand AND and OR operations.

1. CREATE DATABASE

MongoDB Query:

```
use CollegeStudents
```

Explanation: Creates or switches to the CollegeStudents database.



2. CREATE COLLECTION

MongoDB Query:

```
db.createCollection("studentData")
```

Explanation: Creates a collection named studentData.



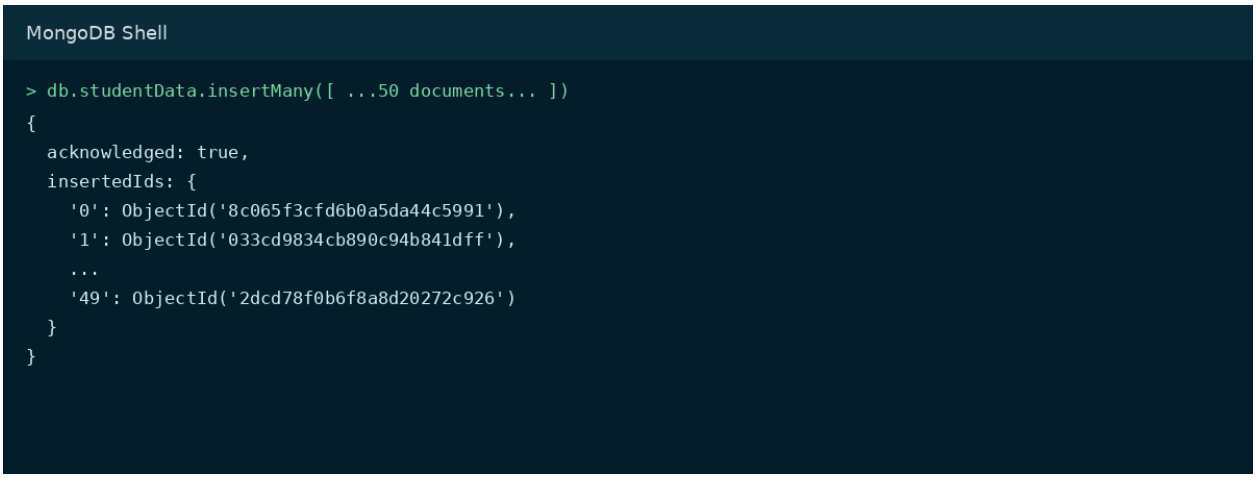
3. INSERT 50 STUDENTS

MongoDB Query:

```
db.studentData.insertMany([
{roll:1, name:"Vihaan", age:19, marks:78, city:"Lucknow"},
{roll:2, name:"Myra", age:18, marks:93, city:"Kanpur"},
{roll:3, name:"Kabir", age:20, marks:81, city:"Varanasi"},
{roll:4, name:"Saanvi", age:19, marks:87, city:"Ayodhya"},
{roll:5, name:"Reyansh", age:18, marks:72, city:"Prayagraj"},
{roll:6, name:"Kiara", age:20, marks:96, city:"Lucknow"},
{roll:7, name:"Ayaan", age:19, marks:69, city:"Gorakhpur"},
{roll:8, name:"Anvi", age:18, marks:85, city:"Jaunpur"},
{roll:9, name:"Atharv", age:20, marks:77, city:"Bareilly"},
{roll:10, name:"Navya", age:19, marks:91, city:"Kanpur"},
{roll:11, name:"Darsh", age:18, marks:63, city:"Lucknow"},
{roll:12, name:"Avni", age:20, marks:88, city:"Varanasi"},
{roll:13, name:"Arnav", age:19, marks:74, city:"Ayodhya"},
{roll:14, name:"Vanya", age:18, marks:95, city:"Prayagraj"},
{roll:15, name:"Dhruv", age:20, marks:82, city:"Lucknow"},
{roll:16, name:"Sara", age:19, marks:67, city:"Gorakhpur"},
{roll:17, name:"Rudra", age:18, marks:89, city:"Jaunpur"},
{roll:18, name:"Mahira", age:20, marks:76, city:"Kanpur"},
{roll:19, name:"Yuvan", age:19, marks:92, city:"Varanasi"},
{roll:20, name:"Aarohi", age:18, marks:71, city:"Lucknow"},
{roll:21, name:"Laksh", age:20, marks:84, city:"Ayodhya"},
{roll:22, name:"Radhika", age:19, marks:79, city:"Prayagraj"},
{roll:23, name:"Om", age:18, marks:86, city:"Lucknow"},
{roll:24, name:"Tia", age:20, marks:65, city:"Gorakhpur"},
{roll:25, name:"Parth", age:19, marks:90, city:"Jaunpur"},
{roll:26, name:"Anushka", age:18, marks:73, city:"Varanasi"},
{roll:27, name:"Raghav", age:20, marks:97, city:"Lucknow"},
{roll:28, name:"Aarav", age:19, marks:68, city:"Kanpur"},
{roll:29, name:"Nitya", age:18, marks:83, city:"Ayodhya"},
{roll:30, name:"Iva", age:20, marks:75, city:"Prayagraj"},
{roll:31, name:"Manav", age:19, marks:94, city:"Lucknow"},
{roll:32, name:"Shanaya", age:18, marks:61, city:"Gorakhpur"},
{roll:33, name:"Kiaan", age:20, marks:80, city:"Jaunpur"},
{roll:34, name:"Aarushi", age:19, marks:88, city:"Varanasi"},
{roll:35, name:"Vivaan", age:18, marks:70, city:"Lucknow"},
{roll:36, name:"Lavanya", age:20, marks:98, city:"Kanpur"},
{roll:37, name:"Rishaan", age:19, marks:66, city:"Ayodhya"},
{roll:38, name:"Meera", age:18, marks:85, city:"Prayagraj"},
{roll:39, name:"Ved", age:20, marks:91, city:"Lucknow"},
{roll:40, name:"Ishaan", age:19, marks:58, city:"Gorakhpur"},
{roll:41, name:"Anaya", age:18, marks:78, city:"Jaunpur"},
{roll:42, name:"Kartik", age:20, marks:93, city:"Varanasi"},
{roll:43, name:"Aadhya", age:19, marks:73, city:"Lucknow"},
{roll:44, name:"Yashvi", age:18, marks:87, city:"Kanpur"},
```

```
{roll:45, name:"Reyansh", age:20, marks:64, city:"Ayodhya"},
{roll:46, name:"Mihir", age:19, marks:81, city:"Prayagraj"},
{roll:47, name:"Inaya", age:18, marks:90, city:"Lucknow"},
{roll:48, name:"Samar", age:20, marks:69, city:"Gorakhpur"},
{roll:49, name:"Prisha", age:19, marks:96, city:"Jaunpur"},
{roll:50, name:"Ronav", age:18, marks:76, city:"Varanasi"},
{roll:51, name:"Ira", age:20, marks:89, city:"Lucknow"}
])
```

Explanation: Inserts 50 new student documents into the studentData collection.



4. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.studentData.find()
```

Explanation: Retrieves documents from the studentData collection.

MongoDB Shell

```
> db.studentData.find()
{
  _id: ObjectId("8c065f3cfd6b0a5da44c5991"),
  roll: 1, name: 'Vihaan', age: 19, marks: 78, city: 'Lucknow'
}
{
  _id: ObjectId("033cd9834cb890c94b841dff"),
  roll: 2, name: 'Myra', age: 18, marks: 93, city: 'Kanpur'
}
{
  _id: ObjectId("9621bae4ad1146cf0c309824"),
  roll: 3, name: 'Kabir', age: 20, marks: 81, city: 'Varanasi'
}
{
  _id: ObjectId("6819bdc43193379cd96c2283"),
  roll: 4, name: 'Saanvi', age: 19, marks: 87, city: 'Ayodhya'
}
{
  _id: ObjectId("efc2eef24f502b0bf5c12e6e"),
  roll: 5, name: 'Reyansh', age: 18, marks: 72, city: 'Prayagraj'
}
{
  _id: ObjectId("d0d07946c71823edf43dc7d9"),
  roll: 6, name: 'Kiara', age: 20, marks: 96, city: 'Lucknow'
}
{
```

5. EQUAL TO — $eq

MongoDB Query:

```
db.studentData.find({marks: {$eq: 93}})
```

Explanation: Finds students whose marks are exactly 93.

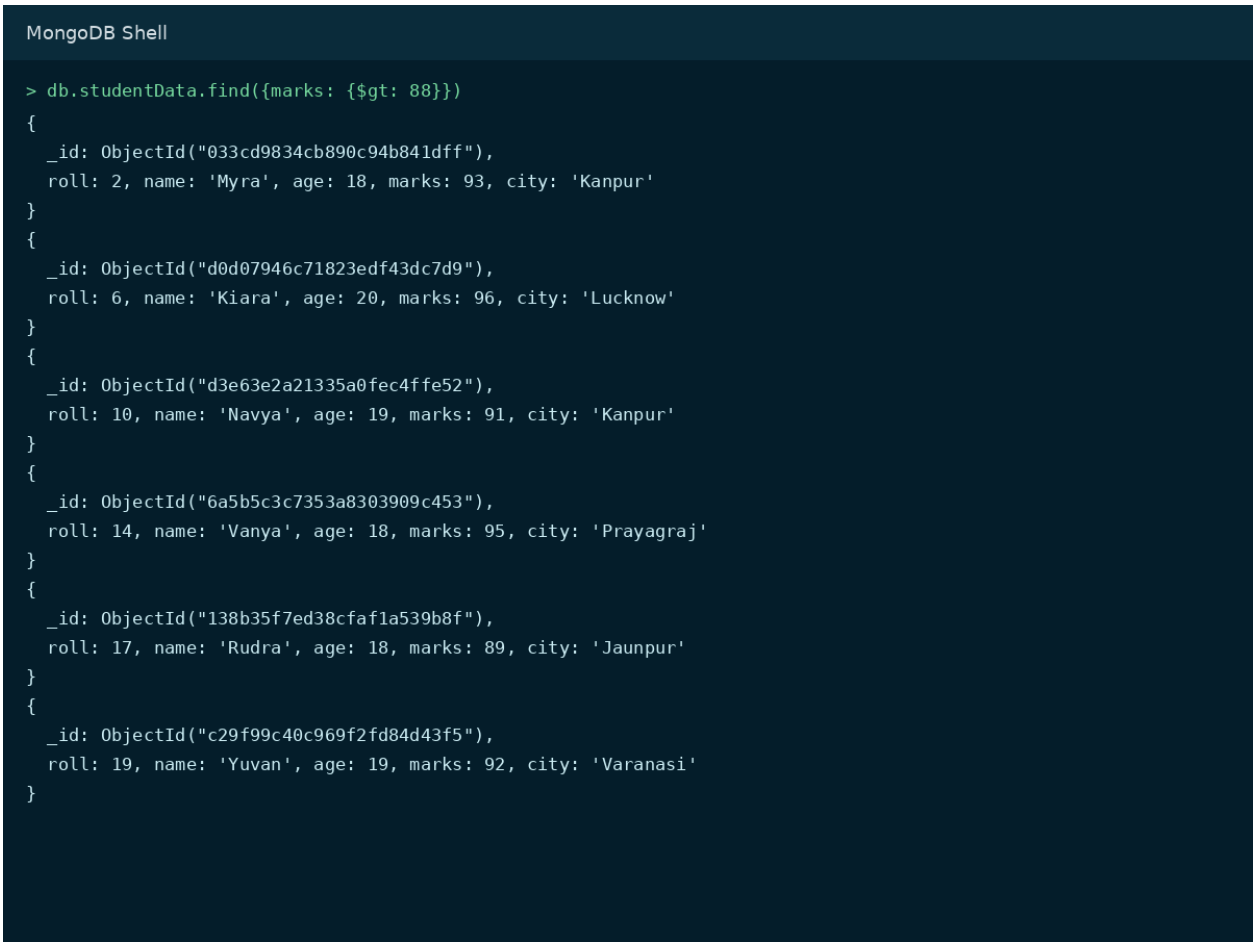


6. GREATER THAN — $gt

MongoDB Query:

```
db.studentData.find({marks: {$gt: 88}})
```

Explanation: Finds students whose marks are greater than 88.



7. LESS THAN — $lt

MongoDB Query:

```
db.studentData.find({marks: {$lt: 65}})
```

Explanation: Finds students whose marks are less than 65.

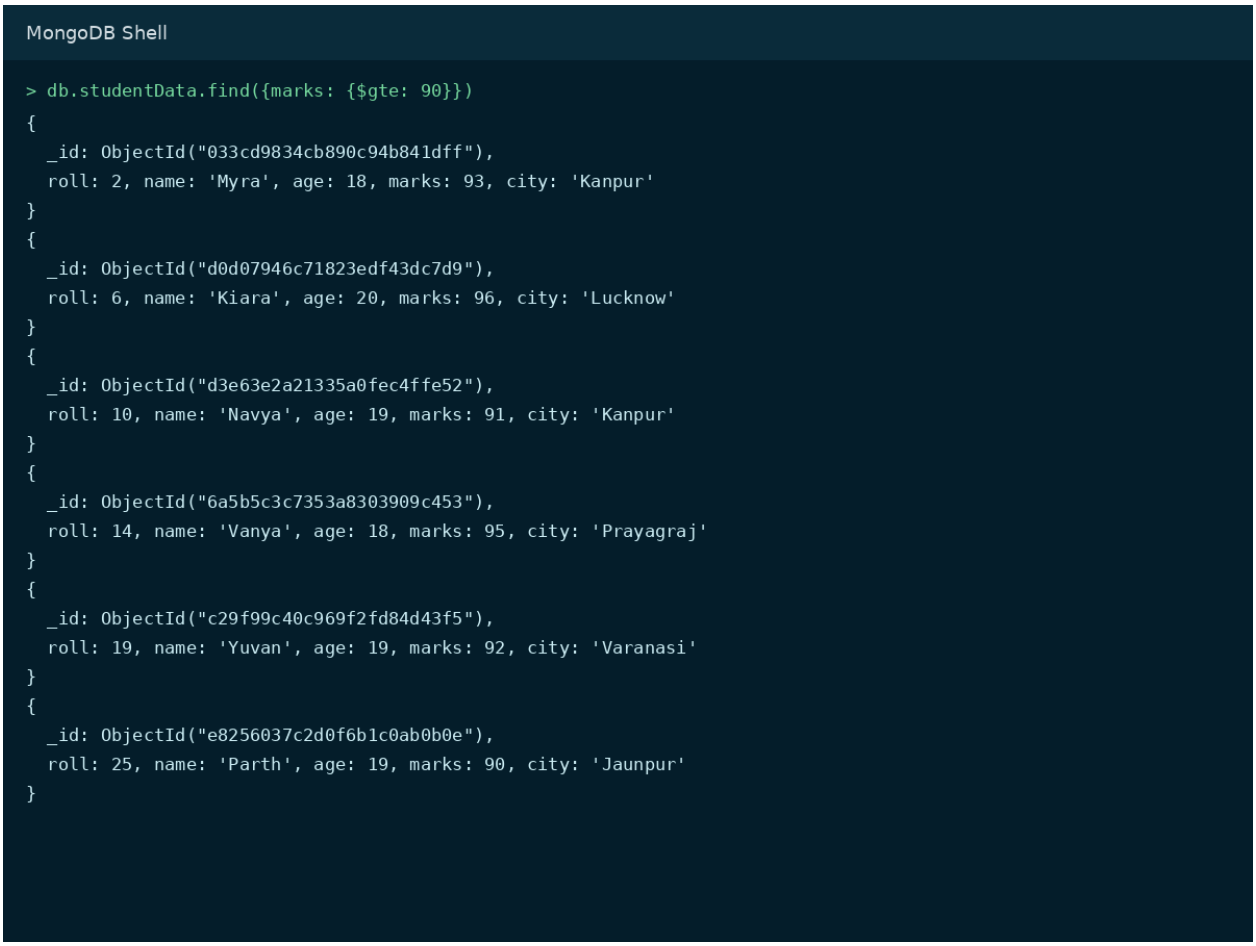


8. GREATER THAN OR EQUAL TO — $gte

MongoDB Query:

```
db.studentData.find({marks: {$gte: 90}})
```

Explanation: Finds students whose marks are 90 or above.



9. LESS THAN OR EQUAL TO — $lte

MongoDB Query:

```
db.studentData.find({marks: {$lte: 70}})
```

Explanation: Finds students whose marks are 70 or below.

MongoDB Shell

```
> db.studentData.find({marks: {$lte: 70}})
{
  _id: ObjectId("c2e57323717301e543869cf8"),
  roll: 7, name: 'Ayaan', age: 19, marks: 69, city: 'Gorakhpur'
}
{
  _id: ObjectId("f74fd9cd87d5e803ac8d04ab"),
  roll: 11, name: 'Darsh', age: 18, marks: 63, city: 'Lucknow'
}
{
  _id: ObjectId("38ee0a3153d837d22911ee8a"),
  roll: 16, name: 'Sara', age: 19, marks: 67, city: 'Gorakhpur'
}
{
  _id: ObjectId("7e955e66845017d932b60c0c"),
  roll: 24, name: 'Tia', age: 20, marks: 65, city: 'Gorakhpur'
}
{
  _id: ObjectId("c03f048edc6d373835b9513e"),
  roll: 28, name: 'Aarav', age: 19, marks: 68, city: 'Kanpur'
}
{
  _id: ObjectId("9f81e64836f493df55fd82f6"),
  roll: 32, name: 'Shanaya', age: 18, marks: 61, city: 'Gorakhpur'
}
```

10. UPDATE OPERATION

MongoDB Query:

```
db.studentData.updateOne(
{roll:4},
{$set:{marks:92}}
)
```

Explanation: Updates the marks to 92 of the student with roll number 4.



11. DELETE ONE OPERATION

MongoDB Query:

```
db.studentData.deleteOne({roll:12})
```

Explanation: Deletes one document matching roll number 12.

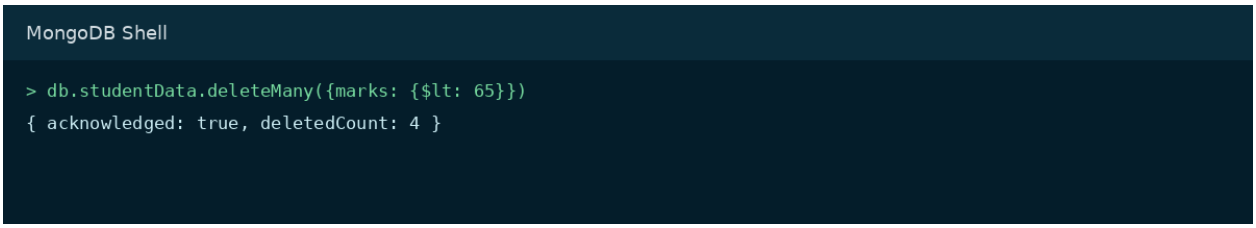


12. DELETE MANY OPERATION

MongoDB Query:

```
db.studentData.deleteMany({marks: {$lt:65}})
```

Explanation: Deletes all students having marks below 65.



13. AND OPERATION — $and

MongoDB Query:

```
db.studentData.find({$and:[{marks:{$gt:80}},{age:19}]})
```

Explanation: Finds students satisfying both conditions.

MongoDB Shell

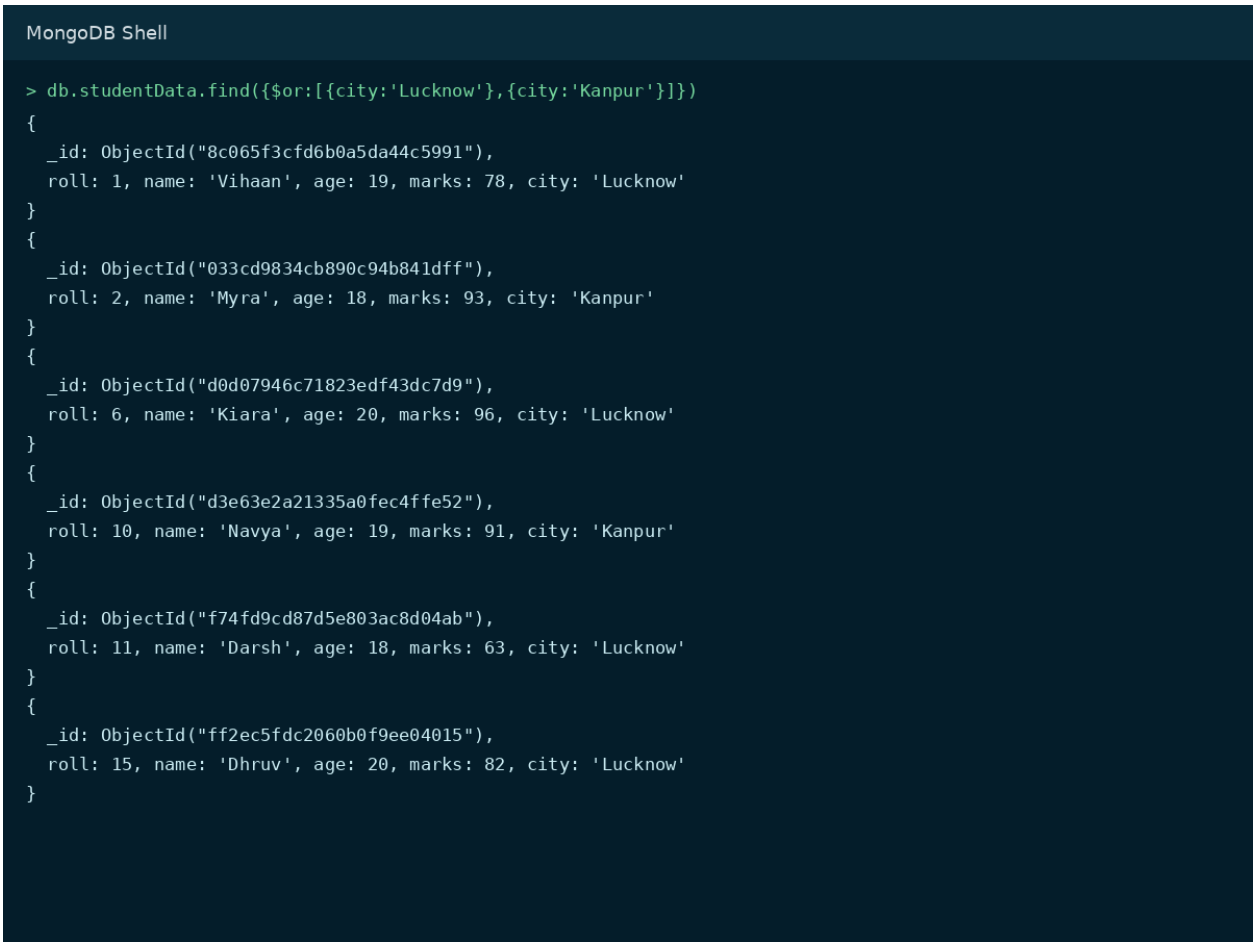
```
> db.studentData.find({$and:[{marks:{$gt:80}},{age:19}]})
{
  _id: ObjectId("6819bdc43193379cd96c2283"),
  roll: 4, name: 'Saanvi', age: 19, marks: 87, city: 'Ayodhya'
}
{
  _id: ObjectId("d3e63e2a21335a0fec4ffe52"),
  roll: 10, name: 'Navya', age: 19, marks: 91, city: 'Kanpur'
}
{
  _id: ObjectId("c29f99c40c969f2fd84d43f5"),
  roll: 19, name: 'Yuvan', age: 19, marks: 92, city: 'Varanasi'
}
{
  _id: ObjectId("e8256037c2d0f6b1c0ab0b0e"),
  roll: 25, name: 'Parth', age: 19, marks: 90, city: 'Jaunpur'
}
{
  _id: ObjectId("c2d563004b2488d6e850ea85"),
  roll: 31, name: 'Manav', age: 19, marks: 94, city: 'Lucknow'
}
{
  _id: ObjectId("fc4182e19e94ffc10c2e1742"),
  roll: 34, name: 'Aarushi', age: 19, marks: 88, city: 'Varanasi'
}
```

14. OR OPERATION — $or

MongoDB Query:

```
db.studentData.find({$or:[{city:'Lucknow'},{city:'Kanpur'}]})
```

Explanation: Finds students from Lucknow or Kanpur.

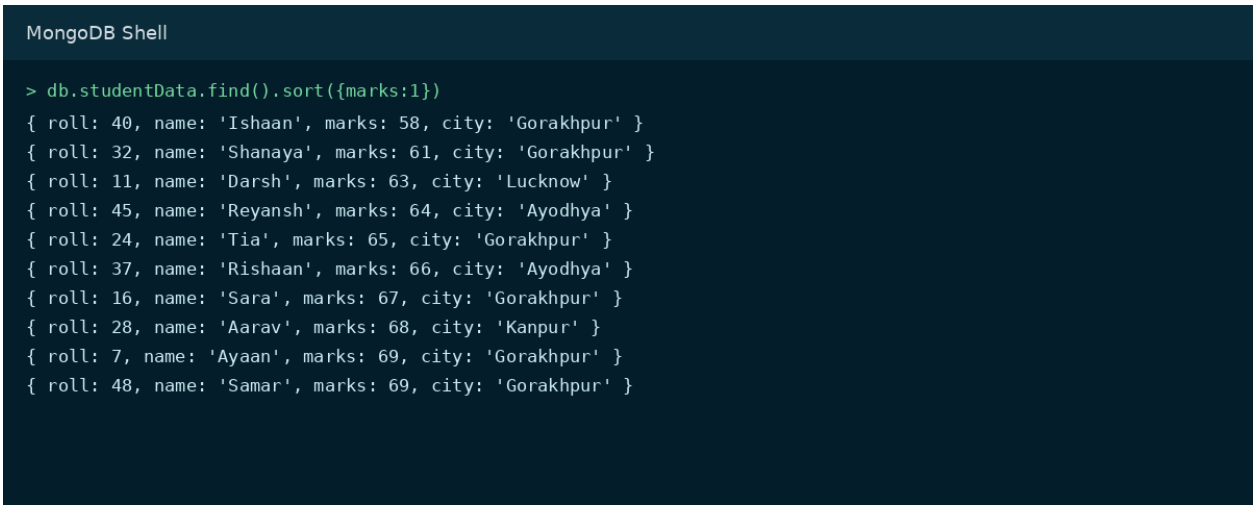


15. SORT OPERATION — ASCENDING

MongoDB Query:

```
db.studentData.find().sort({marks:1})
```

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.



15. SORT OPERATION — DESCENDING

MongoDB Query:

```
db.studentData.find().sort({marks:-1})
```

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

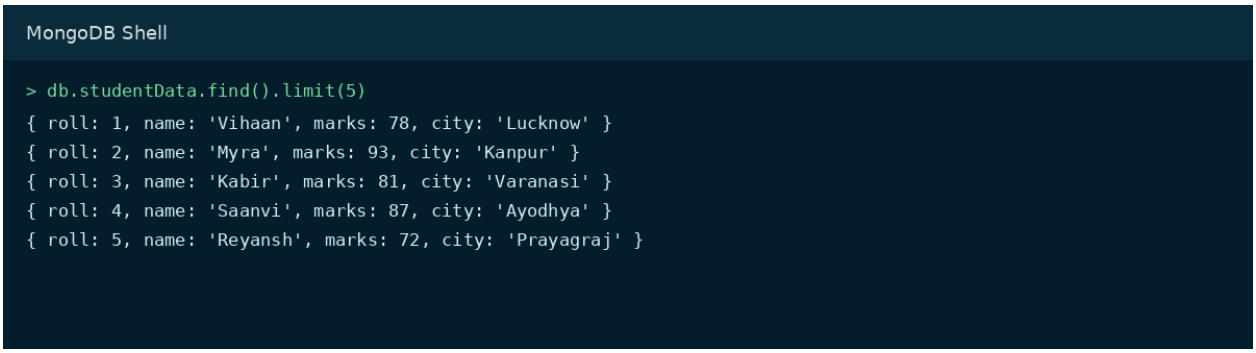


16. LIMIT OPERATION

MongoDB Query:

```
db.studentData.find().limit(5)
```

Explanation: Displays only the first five documents.

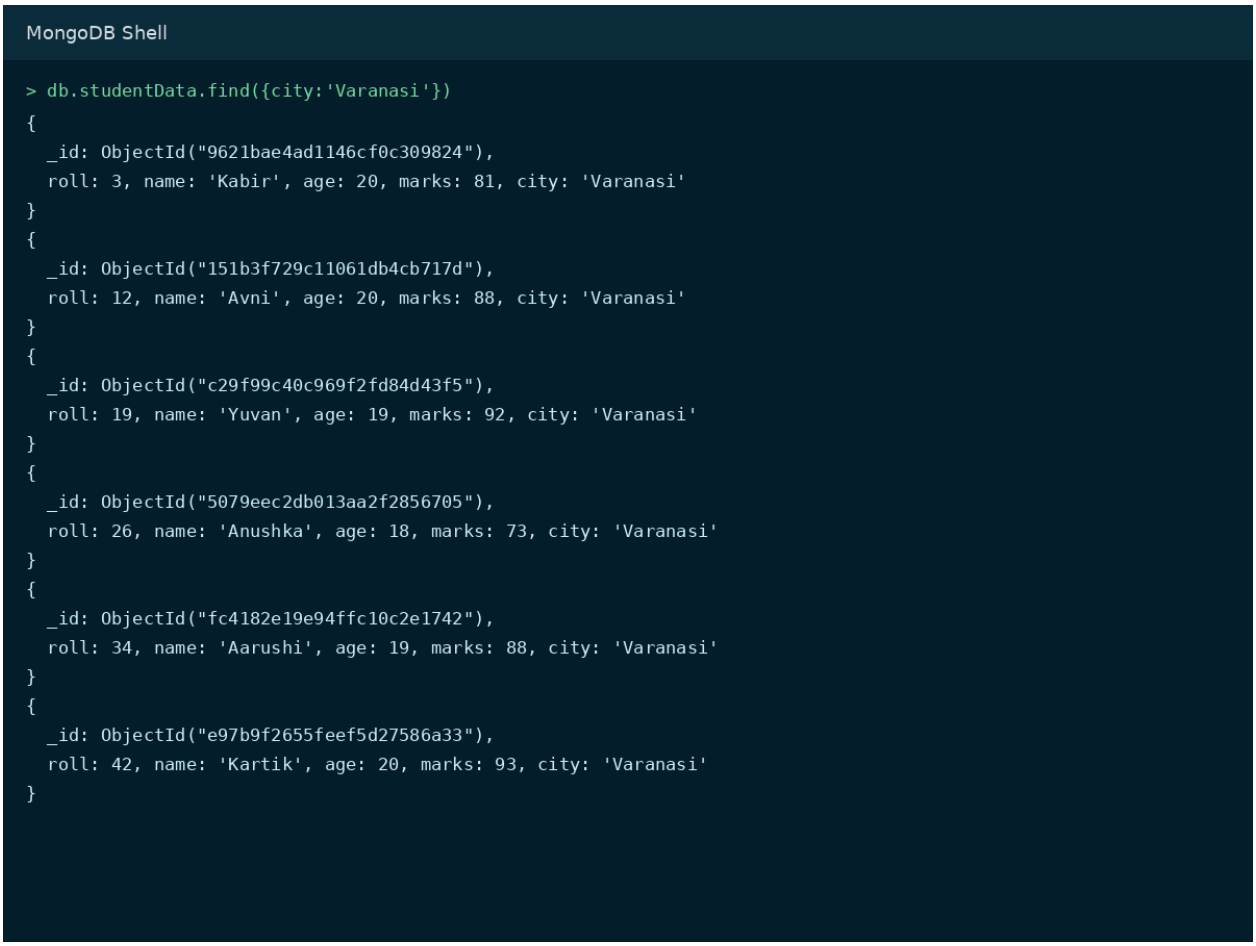


17. SEARCH BY CITY

MongoDB Query:

```
db.studentData.find({city:'Varanasi'})
```

Explanation: Finds all students whose city is Varanasi.



18. COUNT DOCUMENTS

MongoDB Query:

```
db.studentData.countDocuments()
```

Explanation: Counts documents currently present in the collection after the delete operations.



MONGODB OPERATORS USED

Operator	Meaning	Example
$eq	Equal to	marks: {$eq:93}
$gt	Greater than	marks: {$gt:88}
$lt	Less than	marks: {$lt:65}
$gte	Greater than or equal to	marks: {$gte:90}
$lte	Less than or equal to	marks: {$lte:70}
$and	All conditions true	$and:[...]
$or	At least one condition true	$or:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A new Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how ascending and descending order work, comparison operators such as $eq, $gt, $lt, $gte, and $lte work, along with logical operators such as $and and $or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.